Security Audit Report — MEC Frontend (CampusOne)

Date: March 5, 2026 **Application:** MEC Frontend — React Native Mobile App **Version:** 1.0.0 **Auditor:** Security Hardening Review **Scope:** Client-side code vulnerabilities in the React Native codebase

---

Executive Summary

A security audit identified **17 vulnerabilities** across the MEC Frontend React Native application. Of these, **7 actionable code-level fixes** were implemented, **1 pre-existing type error** was resolved, and **5 items** were deferred as they require backend changes, native platform configuration, or are feature additions rather than vulnerability fixes.

Risk Level Before: HIGH **Risk Level After:** MODERATE (remaining items require backend/native work)

Vulnerabilities Fixed

1. Insecure Token Storage

Severity	CRITICAL
File	`src/services/api.ts`
Risk	Authentication tokens (access + refresh) were stored in AsyncStorage, which is unencrypted plaintext on both iOS and Android. Any app with root/jailbreak access or a backup extraction could read tokens and hijack user sessions.
Fix	Migrated token storage from `AsyncStorage` to `react-native-keychain`. Tokens are now stored in the iOS Keychain and Android Keystore, both of which provide hardware-backed encryption. Both tokens are serialized as JSON in a single keychain entry under service `com.campusone.tokens`.
API Preserved	`getAccessToken()`, `getRefreshToken()`, `setTokens()`, `clearTokens()` — all existing callers work without changes.

2. Predictable Device IDs

Severity	MEDIUM
File	`src/store/slices/authSlice.ts`
Risk	Device IDs were generated using `Date.now()` + `Math.random()`, both of which are predictable. An attacker could guess or brute-force device IDs to impersonate devices, potentially bypassing device-based session controls.
Fix	Replaced with `react-native-uuid` (UUID v4), which uses cryptographically secure random number generation. Format: `mobile-<uuid-v4>`. The device ID is still persisted in AsyncStorage (it is not sensitive data, just needs to be unpredictable).

3. User Enumeration via Error Messages

Severity	MEDIUM
File	`src/store/slices/authSlice.ts`
Risk	Different error messages for HTTP 401 ("Invalid username or password") vs 404 ("Account not found") allowed attackers to enumerate valid usernames/accounts. Similarly, the OTP flow revealed whether a phone number was registered.

Fix (Login)	Merged HTTP 401 and 404 responses into a single generic message: **"Invalid credentials. Please try again."** Removed direct passthrough of server error messages for these status codes.
Fix (Send OTP)	HTTP 404 now returns the generic **"Failed to send OTP. Please try again."** instead of "Account not found."
Preserved	HTTP 403 (account approval status), 409 (conflict/duplicate), and 429 (rate limit) messages remain specific as they do not leak enumeration information.

4. Hardcoded Infrastructure URLs in Source Code

Severity	LOW
File	`src/services/api.ts`
Risk	Comment block contained internal IP address (`192.168.1.8`), development URL, and staging URL. If source code is leaked or decompiled, attackers gain knowledge of internal infrastructure topology, making targeted attacks easier.
Fix	Removed the entire comment block containing local IP, dev, and staging URLs. Only the production origin constant remains.

5. Empty Razorpay Key Fallback

Severity	HIGH
File	`src/services/walletService.ts`
Risk	If the backend failed to return a Razorpay `keyId`, the app silently proceeded with an empty string as the payment key. This could cause payment failures that are difficult to debug, or in a worst-case scenario, could be exploited to manipulate payment flows.
Fix	Removed the `RAZORPAY_KEY_FALLBACK` constant entirely. The app now throws an explicit error — **"Razorpay key not returned by server. Cannot proceed with payment."** — if the backend does not return a valid key. This fails fast and visibly rather than silently proceeding.

6. FCM Device ID Leaking User ID

Severity	MEDIUM
File	`src/services/notificationService.ts`
Risk	The FCM token registration sent `deviceId: \${Platform.OS}-${userId}``, directly exposing the internal user ID to the push notification registration endpoint. If FCM token registration data is logged or intercepted, user IDs are leaked. This also creates inconsistency — the device ID used for notifications differed from the one used for authentication.
Fix	Now imports and reuses the same `getDeviceId()` function from `authSlice`, which returns the persisted cryptographically-secure UUID. This ensures consistency across all API calls and eliminates user ID exposure.

7. Unvalidated Socket Notification Payloads

Severity	MEDIUM
File	`src/services/socketService.ts`

Risk (Payload Injection)	The `notification` event handler accepted any payload shape without validation. A compromised or malicious server/MITM could send crafted payloads with missing fields (causing runtime crashes), excessively long strings (UI overflow, memory exhaustion), or injected content (XSS via notification display).
Fix (Validation)	Added a type guard that verifies `title` and `message` are strings before dispatching. Invalid payloads are logged in development and silently dropped in production.
Fix (Truncation)	Title is truncated to 200 characters, message to 1000 characters, preventing UI overflow and memory abuse.
Risk (Stale Auth)	On logout, `disconnectSocket()` cleared the socket reference but left the `auth` object containing the JWT token intact. If the socket object was referenced elsewhere before garbage collection, the stale token could be reused.
Fix (Auth Cleanup)	`disconnectSocket()` now clears `socket.auth = {}` before calling `socket.disconnect()`, ensuring no stale token remains on the socket object.

Additional Fix

8. Type Safety Error in CaptainEatScreen

Severity	LOW (Build Quality)
File	`src/screens/captain/CaptainEatScreen.tsx`
Issue	`onOrderSuccess` callback received a `CreateOrderResult` object but passed it directly to `setSuccessOrder(Order
Fix	Changed to extract `result.order` from the `CreateOrderResult` before setting state.

Deferred Items (Require Backend/Native Changes)

Item	Severity	Reason for Deferral
Certificate Pinning	HIGH	Requires native configuration changes in `Info.plist` (iOS) and `network_security_config.xml` (Android). Not a simple code-level fix. Recommended as immediate follow-up.
Root/Jailbreak Detection	MEDIUM	Requires native package (`jail-monkey` or similar) plus policy decisions on how to handle detected devices. Recommended as follow-up.
Biometric Authentication	LOW	Feature addition (not a vulnerability fix). Enhances UX security but not a current exploit vector.
QR Code HMAC Signing	MEDIUM	Must be implemented server-side to be meaningful. Client-side signing is security theater since the signing key would be bundled in the app. The `pickupToken` from the backend is already the authorization mechanism.
Navigation Parameter Exposure	LOW	Phone numbers and session IDs passed via React Navigation params are in-memory only, not persisted. The app does not use deep linking, so these are not accessible externally.

Files Modified

File	Changes

`package.json`	Added `react-native-keychain`, `react-native-uuid`
`src/services/api.ts`	Keychain token storage, removed hardcoded URLs
`src/store/slices/authSlice.ts`	UUID device IDs, generic error messages, exported `getDeviceId`
`src/services/notificationService.ts`	Consistent device ID for FCM registration
`src/services/walletService.ts`	Removed empty Razorpay key fallback, added error throw
`src/services/socketService.ts`	Notification payload validation, truncation, auth cleanup
`src/screens/captain/CaptainEatScreen.tsx`	Fixed type error in order success handler

Dependencies Added

Package	Version	Purpose
`react-native-keychain`	latest	Secure token storage via iOS Keychain / Android Keystore
`react-native-uuid`	latest	Cryptographically secure UUID v4 generation

Verification Checklist

- [x] `npx tsc --noEmit` passes with zero errors
- [ ] iOS build succeeds (`npm run ios`)
- [ ] Android build succeeds (`npm run android`)
- [ ] Login flow works (tokens stored in Keychain, not AsyncStorage)
- [ ] OTP flow works (send + verify)
- [ ] Wrong credentials show generic "Invalid credentials" message
- [ ] Wrong phone number for OTP shows generic "Failed to send OTP" message
- [ ] Wallet top-up works (Razorpay key validated)
- [ ] Push notifications register with correct device ID
- [ ] Socket notifications display correctly
- [ ] Logout clears Keychain tokens and socket auth

Recommendations for Next Phase

1. **Certificate Pinning** — Implement SSL pinning using `TrustKit` (iOS) and `network_security_config.xml` (Android) to prevent MITM attacks.
2. **Root/Jailbreak Detection** — Add runtime detection with graceful degradation or blocking based on risk tolerance.
3. **Server-Side QR Signing** — Backend should HMAC-sign QR pickup tokens with a server secret and include a timestamp to prevent replay attacks.
4. **Rate Limiting Audit** — Verify all auth endpoints have proper rate limiting on the backend.
5. **Dependency Audit** — Run `npm audit fix` to address 5 known vulnerabilities in transitive dependencies.

Report generated as part of the MEC Frontend security hardening initiative.